



04

# **Classification models**

# Classification Models

## Decision Trees

Decision trees are used in classification tasks, that is, assigning objects to predefined, mutually exclusive categories

### Examples:

**Customer churn cases.** All customers are classified into those likely to stay in the company and those likely to leave

**Medical diagnoses.** Given a set of symptoms for a patient, classify the patient according to a disease

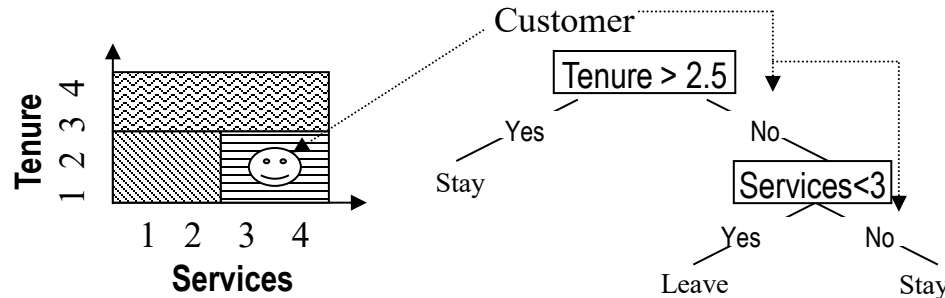
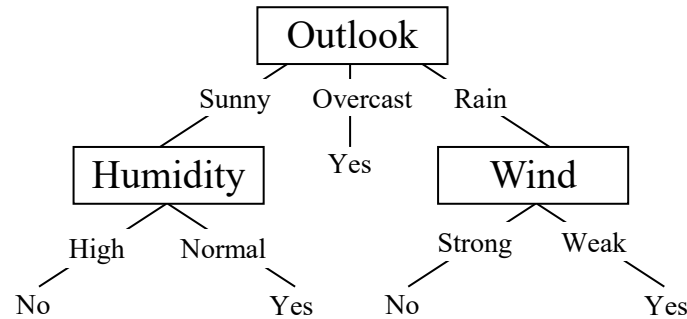
**Play tennis.** Days are classified into “good for playing tennis” and “not good”

# Classification Models

## Decision Trees

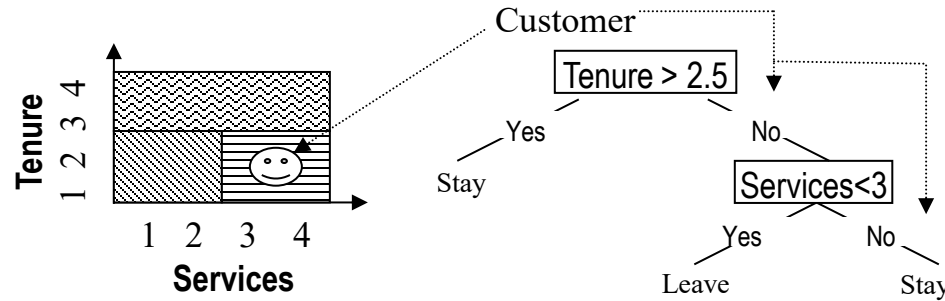
The predictive model is in the form of a decision tree, often a binary tree.

Examples:



# Classification Models

## Decision Trees



In the customer churn example, the most important variable is the number of years that the customer has been with the company (Tenure) .

The tree divides the client database into two parts – those who have been with the company for more than two and one-half years and those who have not.

# Classification Models

## Decision Trees

### How to build a decision tree?

Many **methods** are reported in the literature. Important contributions from at least three disciplines;

- Statistics → Recursive Partitioning Methods
- Machine Learning → Induction of decision trees
- Pattern Recognition → Construction of classification trees

And many **systems** too!!

# Decision Trees

## C4.5

**C4.5** belongs to a family of decision-tree learners that trace their origins back to the work of Hunt (1962). Its immediate predecessors were **ID3** (Quinlan, 1979), a simple system consisting of about 600 lines of Pascal, and **C4** (Quinlan, 1987).

C4.5 has grown to about 9,000 lines of C that is available on floppy disk with Quinlan (1993). It has been superseded by **C5.0**, a commercial system from RuleQuest Research ([www.rulequest.com](http://www.rulequest.com)).

Hunt, E.B. (1962). *Concept Learning: An Information Processing Problem*. New York, NY: Wiley

Quinlan, J.R. (1979). Discovering rules from large collections of examples: a case study. *Expert systems in the Micro Electronic Age*, D. Michie (Ed.), Scotland: Edimburgh University Press

Quinlan, J.R. (1987). Inductive knowledge acquisition: A case study. *Applications of Expert Systems*, Quinlan (Ed.), pp. 157-173, Wokigham, England: Addison-Wesley.

Quinlan, J.R. (1993). *C4.5: Programs for machine learning*. San Mateo, CA: Morgan Kaufman.

# Decision Trees

## C4.5: Input

Input: a collection of training examples, each having a tuple of values for a fixed **set of attributes** (or **independent variables**)

$$A = \{A_1, A_2, \dots, A_m\}$$

and a **class attribute** (or **dependent variable**).

An attribute  $A_i$  is described as **continuous** or **discrete** depending on whether its values are numeric or nominal.

The class attribute  $C$  is discrete and has values  $C_1, C_2, \dots, C_k$ .

# Decision Trees

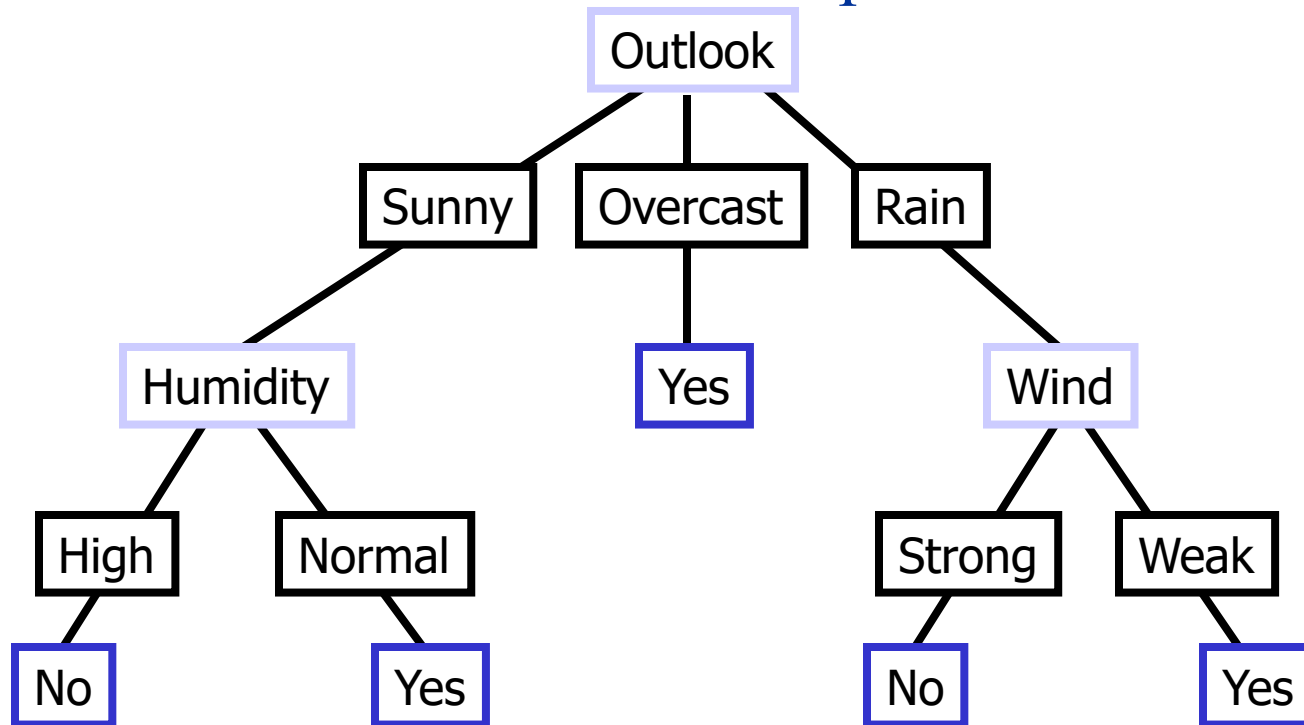
## C4.5: Input

| Day | Outlook  | Temperature | Humidity | Wind   | PlayTennis |
|-----|----------|-------------|----------|--------|------------|
| D1  | Sunny    | Hot         | High     | Weak   | No         |
| D2  | Sunny    | Hot         | High     | Strong | No         |
| D3  | Overcast | Hot         | High     | Weak   | Yes        |
| D4  | Rain     | Mild        | High     | Weak   | Yes        |
| D5  | Rain     | Cool        | Normal   | Weak   | Yes        |
| D6  | Rain     | Cool        | Normal   | Strong | No         |
| D7  | Overcast | Cool        | Normal   | Strong | Yes        |
| D8  | Sunny    | Mild        | High     | Weak   | No         |
| D9  | Sunny    | Cool        | Normal   | Weak   | Yes        |
| D10 | Rain     | Mild        | Normal   | Weak   | Yes        |
| D11 | Sunny    | Mild        | Normal   | Strong | Yes        |
| D12 | Overcast | Mild        | High     | Strong | Yes        |
| D13 | Overcast | Hot         | Normal   | Weak   | Yes        |
| D14 | Rain     | Mild        | High     | Strong | No         |

[See: Tom M. Mitchell, *Machine Learning*, McGraw-Hill, 1997]

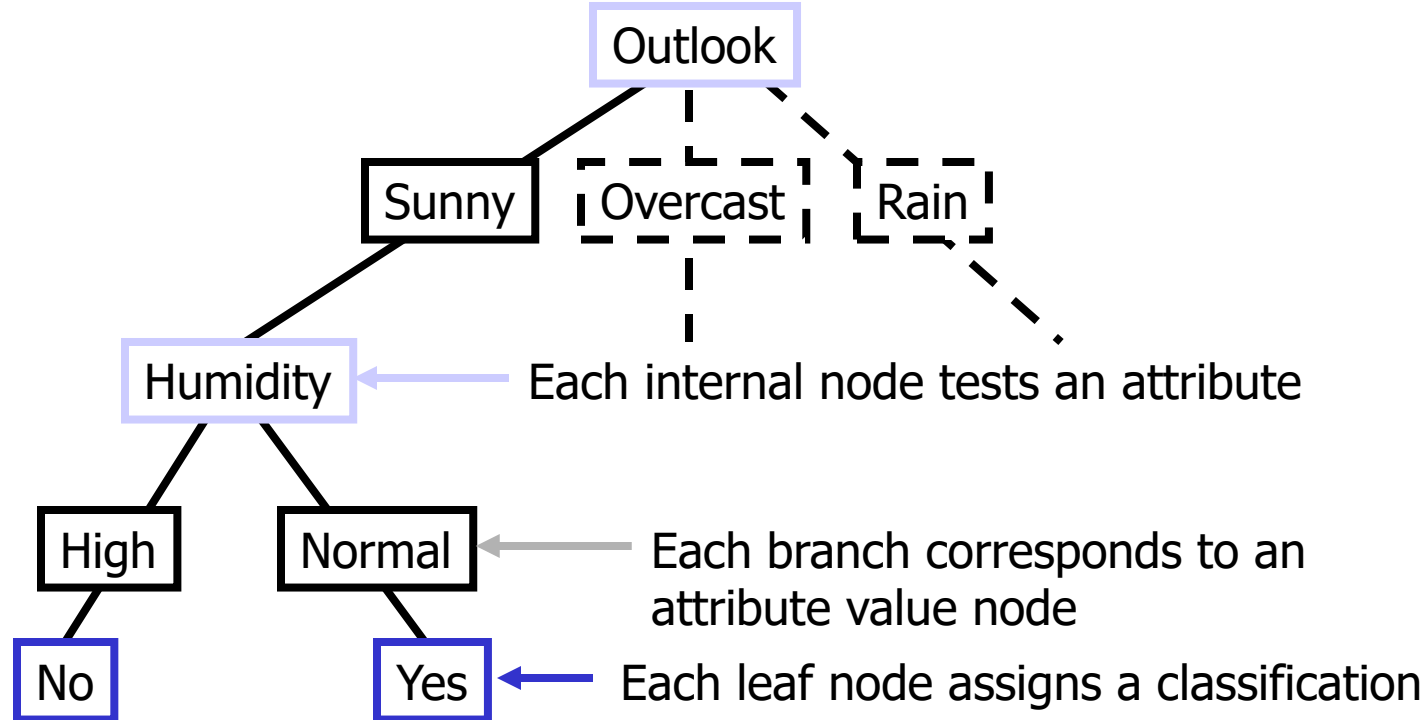
# Decision Trees

## C4.5: Output



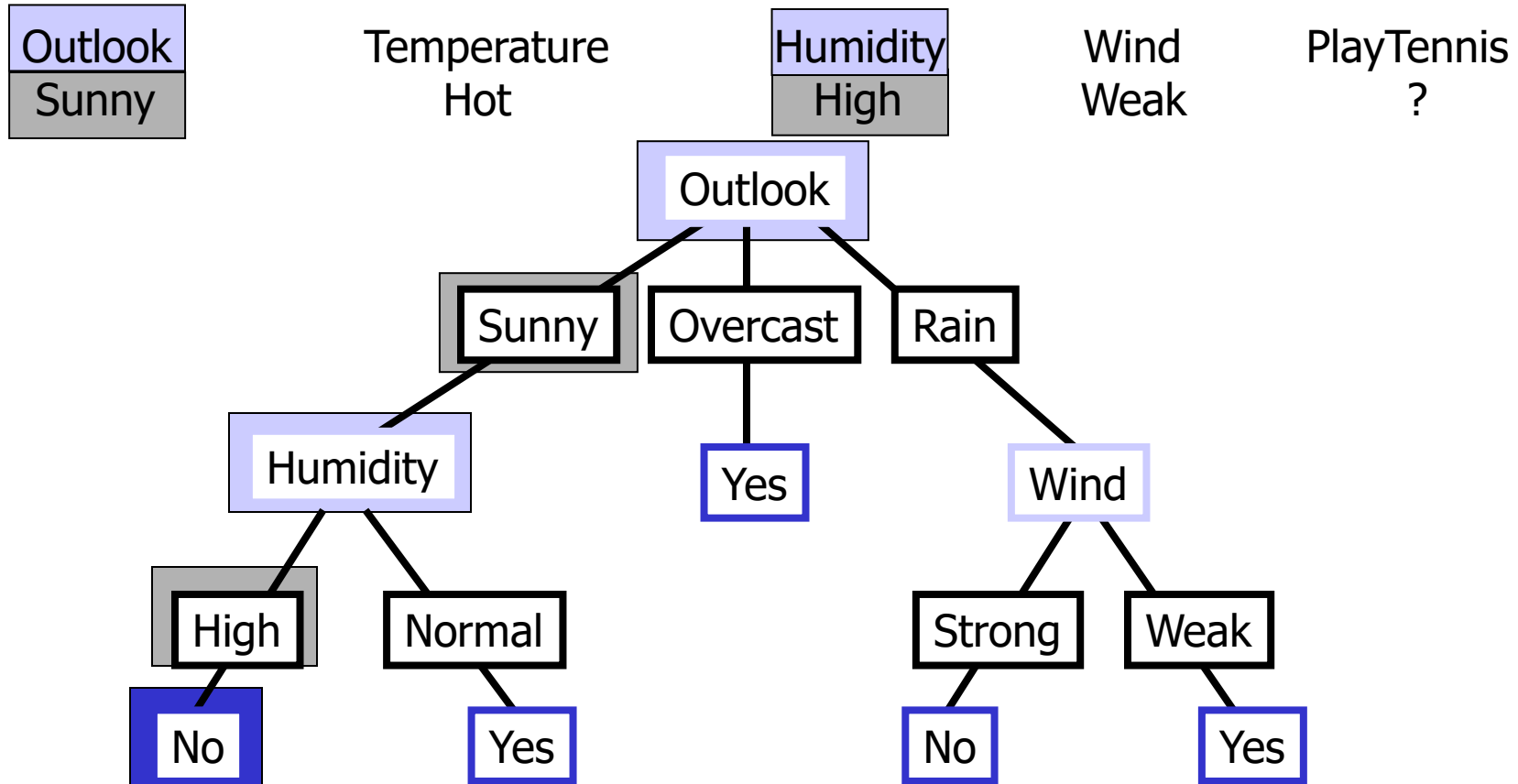
# Decision Trees

## C4.5: Output



# Decision Trees

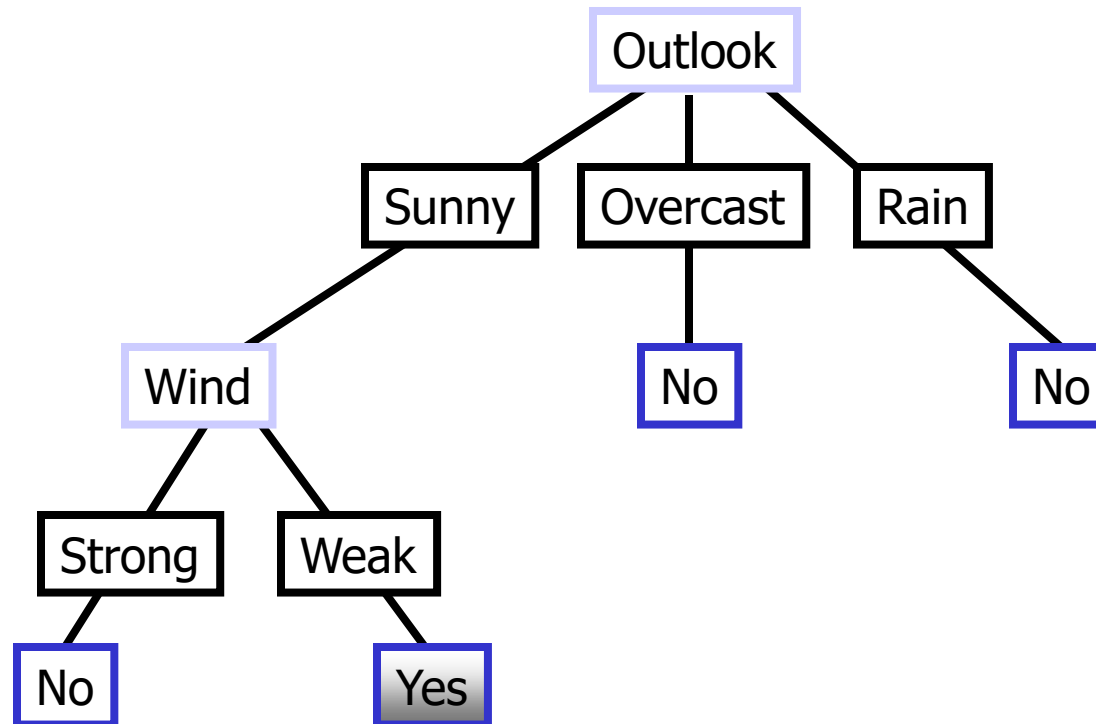
## How to use the tree?



# Decision Trees

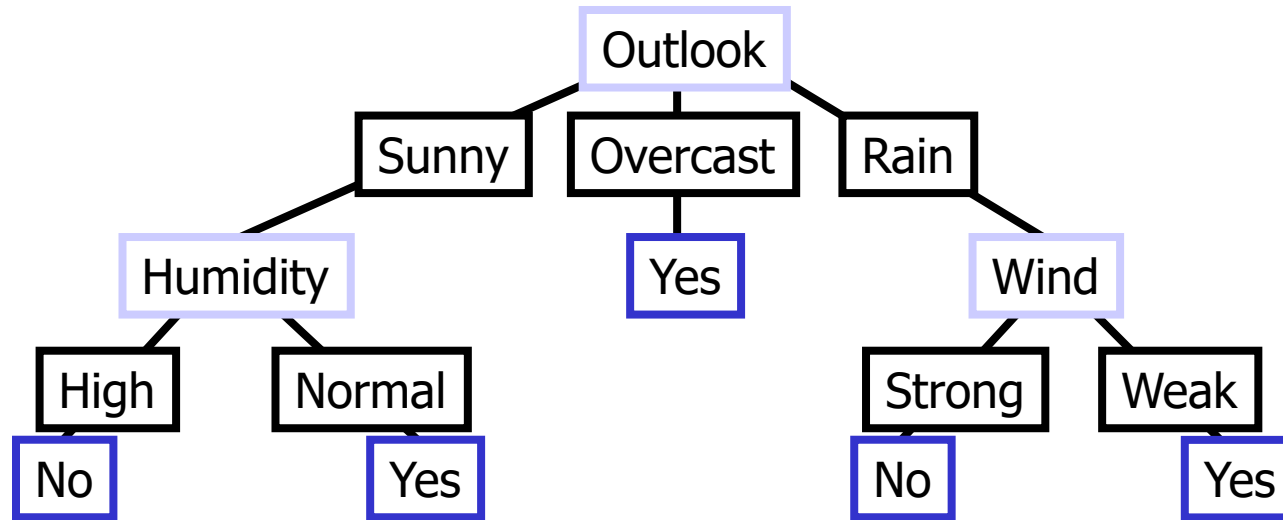
## Decision tree for Conjunction

Outlook=Sunny  $\wedge$  Wind=Weak



# Decision Trees

Represent disjunctions of conjunctions



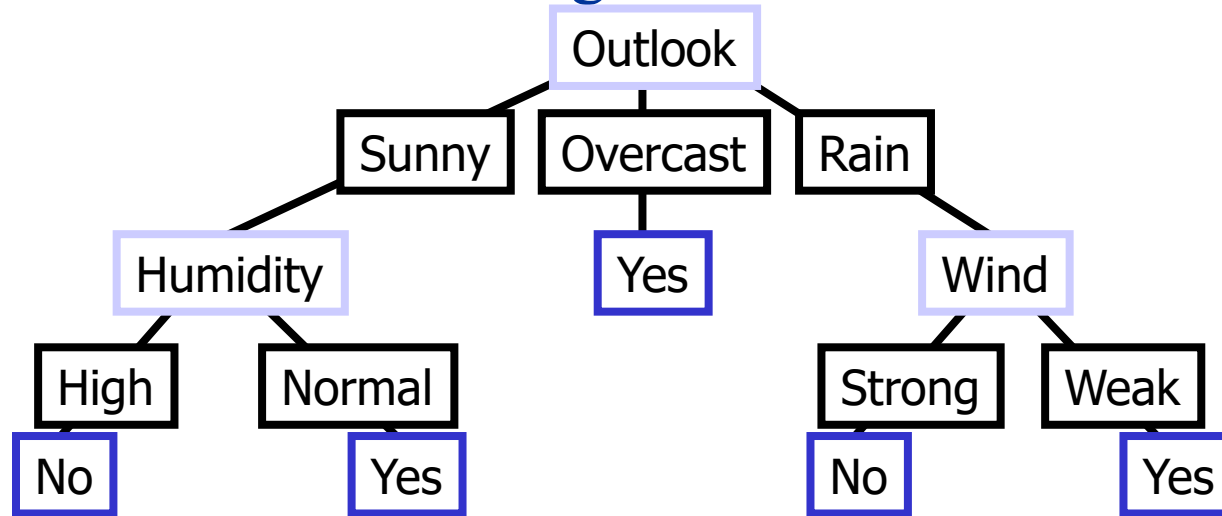
(Outlook=Sunny  $\wedge$  Humidity=Normal)

✓ (Outlook=Overcast)

✓ (Outlook=Rain  $\wedge$  Wind=Weak)

# Decision Trees

## Converting a tree to rules



$R_1$ : If (Outlook=Sunny)  $\wedge$  (Humidity=High) Then PlayTennis=No

$R_2$ : If (Outlook=Sunny)  $\wedge$  (Humidity=Normal) Then PlayTennis=Yes

$R_3$ : If (Outlook=Overcast) Then PlayTennis=Yes

$R_4$ : If (Outlook=Rain)  $\wedge$  (Wind=Strong) Then PlayTennis=No

$R_5$ : If (Outlook=Rain)  $\wedge$  (Wind=Weak) Then PlayTennis=Yes

# Decision Trees

## C4.5: Input with numerical attributes

**Sample Names File:** *golf.names*

```
Play, Don't Play.
```

```
outlook: sunny, overcast, rain.  
temperature: continuous.  
humidity: continuous.  
windy: true, false.
```

**Sample Data File:** *golf.data*

```
Sunny, 85, 85, false, Don't Play  
sunny, 80, 90, true, Don't Play  
overcast, 83, 78, false, Play  
rain, 70, 96, false, Play  
rain, 68, 80, false, Play  
rain, 65, 70, true, Don't Play  
overcast, 64, 65, true, Play  
sunny, 72, 95, false, Don't Play  
sunny, 69, 70, false, Play  
rain, 75, 80, false, Play  
sunny, 75, 70, true, Play  
overcast, 72, 90, true, Play  
overcast, 81, 75, false, Play  
rain, 71, 80, true, Don't Play
```

# Decision Trees

## C4.5: Output with numerical attributes

4.5 [release 8] decision tree generator Thu Jun 15 09:15:50 2000

Options:  
File stem <golf>

Read 14 cases (4 attributes) from golf.data

Decision Tree:

```

outlook = overcast: Play (4.0)
outlook = sunny:
|  humidity <= 75 : Play (2.0)
|  humidity > 75 : Don't Play (3.0)
outlook = rain:
|  windy = true: Don't Play (2.0)
|  windy = false: Play (3.0)

```

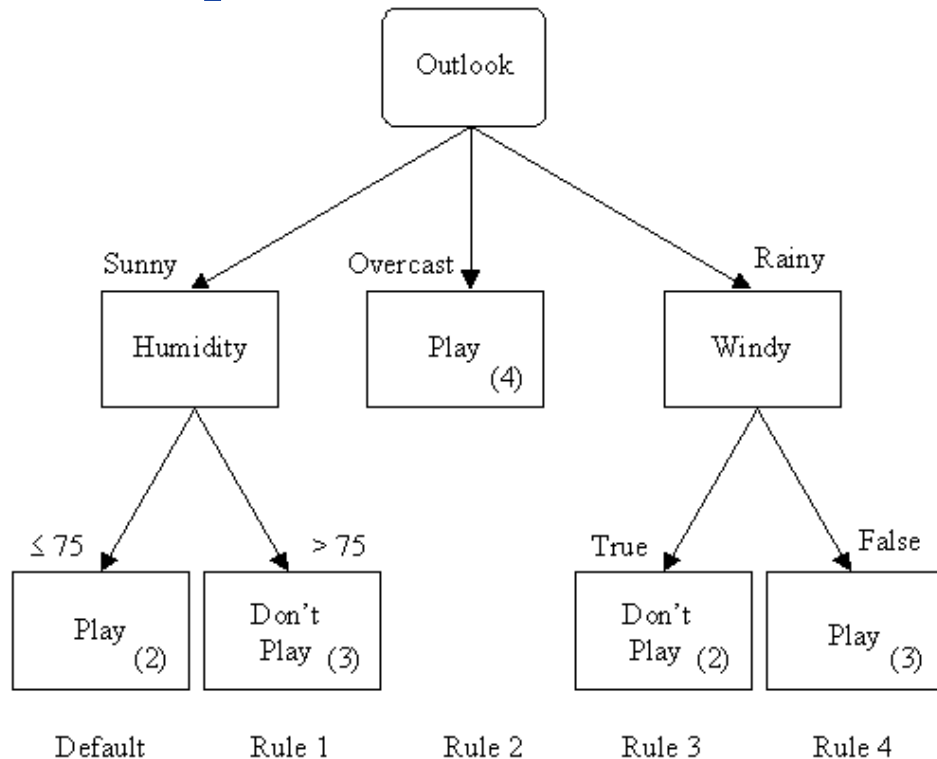
Tree saved

Evaluation on training data (14 items):

| Before Pruning |          | After Pruning |          |          |    |
|----------------|----------|---------------|----------|----------|----|
| Size           | Errors   | Size          | Errors   | Estimate |    |
| 8              | 0( 0.0%) | 8             | 0( 0.0%) | (38.5%)  | << |

# Decision Trees

## C4.5: Output with numerical attributes



C4.5 [release 8] rule generator Thu Jun 15 09:15:54 2000

Options:  
File stem <golf>

Read 14 cases (4 attributes) from golf

Processing tree 0

Final rules from tree 0:

Rule 2:  
outlook = overcast  
-> class Play [70.7%]

Rule 4:  
outlook = rain  
windy = false  
-> class Play [63.0%]

Rule 1:  
outlook = sunny  
humidity > 75  
-> class Don't Play [63.0%]

Rule 3:  
outlook = rain  
windy = true  
-> class Don't Play [50.0%]

Default class: Play

Evaluation on training data (14 items):

| Rule | Size | Error | Used | Wrong    | Advantage |            |
|------|------|-------|------|----------|-----------|------------|
| 2    | 1    | 29.3% | 4    | 0 (0.0%) | 0 (010)   | Play       |
| 4    | 2    | 37.0% | 3    | 0 (0.0%) | 0 (010)   | Play       |
| 1    | 2    | 37.0% | 3    | 0 (0.0%) | 3 (310)   | Don't Play |
| 3    | 2    | 50.0% | 2    | 0 (0.0%) | 2 (210)   | Don't Play |

Tested 14. errors 0 (0.0%) <<

(a) (b) <-classified as

9 (a): class Play  
5 (b): class Don't Play

# Decision Trees

## C4.5: Output Rules

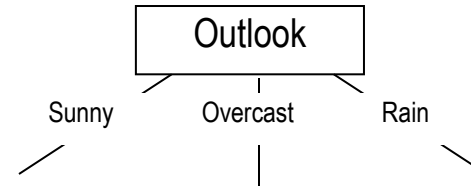
# Decision Trees

## C4.5: Candidate tests

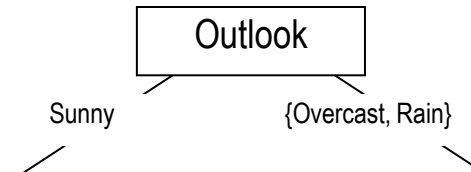
C4.5 uses tests of three types, each involving a single attribute  $A_i$ .

If  $A_i$  is discrete, possible tests are

- $z$  outcomes, one for each value of  $A_i$   $\leftarrow$  default



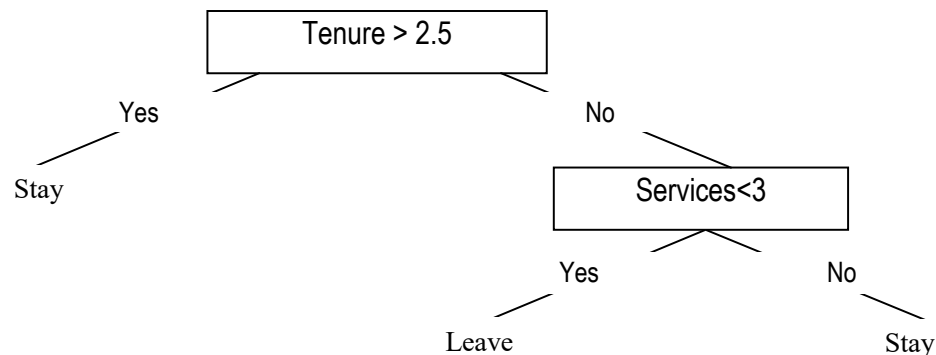
- $2 \leq w \leq z$  outcomes, where  $G = \{G_1, G_2, \dots, G_w\}$  is a partition of values of  $A_i$



# Decision Trees

## C4.5: Candidate tests

If  $A_i$  is continuous, then it is split at some derived threshold value. The for of the test is  $A_i \leq \Theta$  (or  $A_i > \Theta$ ) with outcomes *true* and *false*.



# Decision Trees

## C4.5: Selecting tests

**Question:** How do you determine which attribute best classifies data?

**Answer:** Entropy!!!

Let us consider:

- $S$  the set of training examples currently analysed.
- $C_j$  a class label in  $C_1, C_2, \dots, C_k$ .
- $RF(C_i, S)$ ,  $i=1, \dots, k$  the relative frequencies of cases in  $S$  that belong to class  $C_i$ .

The entropy  $E$  on  $S$  is computed as:

$$E(S) = - \sum_{i=1..k} RF(C_i, S) \log(RF(C_i, S))$$



# Decision Trees

## C4.5: Selecting tests

- Entropy measures the **uncertainty** contained in  $S$ .
  - Takes the largest value when events are equiprobable

$$RF(C_1, S) = \dots = RF(C_k, S),$$

This is a good thing since an ensemble of equally probable events is as uncertain as it gets.

- Takes the smallest value when only one event has non-zero probability.

$$RF(C_i, S) = 1 \quad RF(C_j, S) = 0 \quad \forall j \neq i$$

# Decision Trees

## C4.5: Selecting tests

### RECAP

Uncertainty in a System: Entropy quantifies how unpredictable a system is. If all events are equally likely, the system is highly uncertain, and entropy is maximized.

For example, if you have a fair six-sided die, each face (event) has a probability of  $1/6$ . The entropy is high because any face could come up with equal likelihood.

### Maximum Entropy:

Entropy reaches its maximum value when all events are equally probable. This is because the uncertainty is at its highest; you have no more information about which event will occur.

### Minimum Entropy:

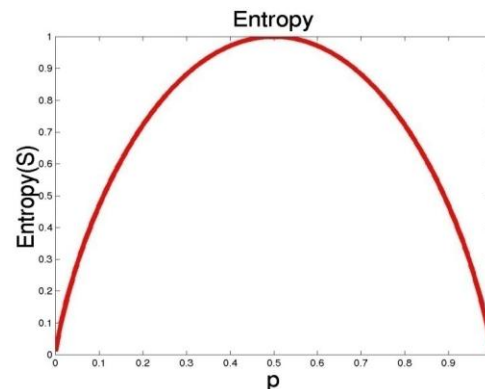
Entropy is at its minimum when one event is certain (probability 1) and all others are impossible (probability 0). In this case, there is no uncertainty because the outcome is known.

High Entropy: Indicates a high level of disorder or unpredictability. Systems with high entropy are less predictable.

Low Entropy: Indicates a low level of disorder or high predictability. Systems with low entropy are more predictable.

# Decision Trees

## C4.5: Selecting tests



In two class problems ( $C_1$ =positive,  $C_2$ =negative) where:

- $p$  is the proportion of positive examples in  $S$
- $n$  is the proportion of negative examples in  $S$

the entropy of  $S$  is computed as:

$$E(S) = -p \log_2 p - n \log_2 n$$

# Decision Trees

## C4.5: Selecting tests

- The **Information Gain**  $G(S,t)$  represents the expected reduction in entropy due to distributing instances on  $S$  according the test  $t$ .
- Let us consider  $S_1, \dots, S_z$  the partitioning of  $S$  by a test  $t$  on an attribute  $A_i$ :

$$G(S,t) = E(S) - \sum_i \frac{|S_i|}{|S|} E(S_i)$$

The information gain criterion chooses the test  $t$  that maximizes  $G(S,t)$

# Decision Trees

## C4.5: Selecting tests

Information Gain is a measure used in decision trees and other machine learning algorithms to determine the effectiveness of a feature (or test) in classifying data. It represents the expected reduction in entropy (uncertainty) when the dataset  $S$  is split according to a particular test  $t$ .

Entropy Before the Split: Initially, we calculate the entropy of the entire dataset  $S$ . This entropy represents the uncertainty or impurity in the dataset before any split is made.

Splitting the Dataset: The dataset  $S$  is then split into subsets based on the outcome of the test  $t$ . Each subset will have its own entropy, which reflects the uncertainty within that subset.

Entropy After the Split: After splitting, we calculate the weighted average of the entropies of the resulting subsets. The weights are proportional to the sizes of the subsets.

Information Gain Calculation: Information Gain is the difference between the entropy of the original dataset and the weighted average entropy of the subsets.

# Decision Trees

## C4.5: Selecting tests

**High Information Gain:** Indicates that the test  $t$  effectively reduces uncertainty, meaning it is a good feature for splitting the data.

**Low Information Gain:** Suggests that the test  $t$  does not significantly reduce uncertainty, meaning it is not a useful feature for splitting the data.

# Decision Trees

## C4.5: Selecting tests

### Example

$$G(S, t) = E(S) - \sum_i \frac{|S_i|}{|S|} E(S_i)$$

$$E(S) = - \sum_{i=1..k} RF(C_i, S) \log(RF(C_i, S))$$

| Day | Outlook  | Temperature | Humidity | Wind   | PlayTennis |
|-----|----------|-------------|----------|--------|------------|
| D1  | Sunny    | Hot         | High     | Weak   | No         |
| D2  | Sunny    | Hot         | High     | Strong | No         |
| D3  | Overcast | Hot         | High     | Weak   | Yes        |
| D4  | Rain     | Mild        | High     | Weak   | Yes        |
| D5  | Rain     | Cool        | Normal   | Weak   | Yes        |
| D6  | Rain     | Cool        | Normal   | Strong | No         |
| D7  | Overcast | Cool        | Normal   | Strong | Yes        |
| D8  | Sunny    | Mild        | High     | Weak   | No         |
| D9  | Sunny    | Cool        | Normal   | Weak   | Yes        |
| D10 | Rain     | Mild        | Normal   | Weak   | Yes        |
| D11 | Sunny    | Mild        | Normal   | Strong | Yes        |
| D12 | Overcast | Mild        | High     | Strong | Yes        |
| D13 | Overcast | Hot         | Normal   | Weak   | Yes        |
| D14 | Rain     | Mild        | High     | Strong | No         |

## Step 1: Calculate the Entropy of the Entire Dataset

We have 14 training examples: 9 are "Yes" and 5 are "No". The entropy  $E(S)$  of the entire dataset is calculated as follows:

$$E(S) = - \left( \frac{9}{14} \right) \log_2 \left( \frac{9}{14} \right) - \left( \frac{5}{14} \right) \log_2 \left( \frac{5}{14} \right)$$

Plugging in the values:

$$E(S) = - \left( \frac{9}{14} \right) \log_2 \left( \frac{9}{14} \right) - \left( \frac{5}{14} \right) \log_2 \left( \frac{5}{14} \right) \approx 0.940$$

The "Outlook" feature has three possible values: sunny, overcast, and rain. We need to calculate the entropy for each subset.

1. **Outlook = Sunny:**

- 5 examples: 2 are "Yes", 3 are "No".
- Entropy  $E(S_{\text{sunny}})$ :

$$E(S_{\text{sunny}}) = - \left(\frac{2}{5}\right) \log_2 \left(\frac{2}{5}\right) - \left(\frac{3}{5}\right) \log_2 \left(\frac{3}{5}\right) \approx 0.971$$

## Step 2: Calculate the Entropy for Each Subset Based on "Outlook"

2. **Outlook = Overcast:**

- 4 examples: 4 are "Yes", 0 are "No".
- Entropy  $E(S_{\text{overcast}})$ :

$$E(S_{\text{overcast}}) = - \left(\frac{4}{4}\right) \log_2 \left(\frac{4}{4}\right) - \left(\frac{0}{4}\right) \log_2 \left(\frac{0}{4}\right)$$

Since  $\log_2(1) = 0$  and  $\log_2(0)$  is undefined but multiplied by 0, it is 0:

$$E(S_{\text{overcast}}) = 0$$

### 3. Outlook = Rain:

- 5 examples: 3 are "Yes", 2 are "No".
- Entropy  $E(S_{rain})$ :

$$E(S_{rain}) = - \left(\frac{3}{5}\right) \log_2 \left(\frac{3}{5}\right) - \left(\frac{2}{5}\right) \log_2 \left(\frac{2}{5}\right) \approx 0.971$$

**Step 2: Calculate the Entropy  
for Each Subset Based on  
"Outlook"**

### Step 3: Calculate the Weighted Average Entropy After the Split

We now calculate the weighted average entropy of the subsets:

$$E_{split}(S, \text{Outlook}) = \left(\frac{5}{14}\right) E(S_{sunny}) + \left(\frac{4}{14}\right) E(S_{overcast}) + \left(\frac{5}{14}\right) E(S_{rain})$$

Plugging in the values:

$$E_{split}(S, \text{Outlook}) = \left(\frac{5}{14}\right) (0.971) + \left(\frac{4}{14}\right) (0) + \left(\frac{5}{14}\right) (0.971)$$

$$E_{split}(S, \text{Outlook}) = \left(\frac{5}{14}\right) (0.971) + \left(\frac{5}{14}\right) (0.971)$$

$$E_{split}(S, \text{Outlook}) = \left(\frac{10}{14}\right) (0.971) \approx 0.693$$

## Step 4: Calculate the Information Gain

Finally, the Information Gain  $G(S, \text{Outlook})$  is the difference between the entropy of the original dataset and the weighted average entropy after the split:

$$G(S, \text{Outlook}) = E(S) - E_{split}(S, \text{Outlook})$$

Plugging in the values:

$$G(S, \text{Outlook}) = 0.940 - 0.693 \approx 0.247$$

Similarly, we may compute:

- $G(\text{Temperature})=0.029$
- $G(\text{Humidity})=0.151$
- $G(\text{Windy})=0.048$

**The test that maximizes the information gain is the test performed on *Outlook***

# Decision Trees

## Alternative criteria

Let  $p$  the proportion of positive  
 $1-p$  the proportion of negative examples

(i) Entropy

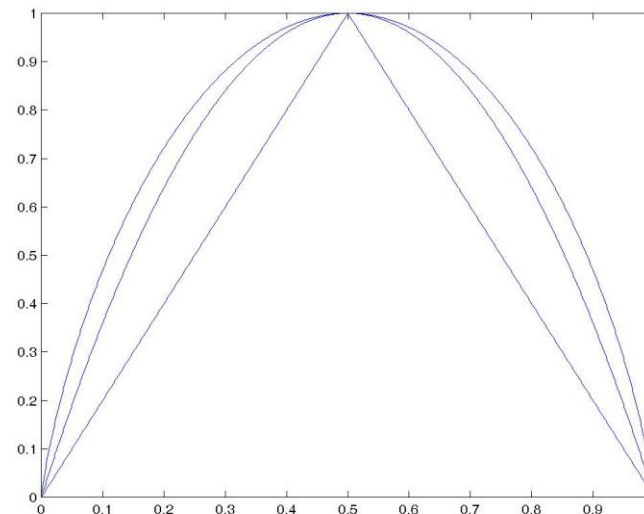
$$p \log \frac{1}{p} + (1-p) \log \frac{1}{1-p}$$

(i) Gini index

$$2p(1-p)$$

(i) “Simplest of all”

$$\min\{p, 1-p\}$$



# Decision Trees

## Alternative criteria

|                          | $k = 2$                                             | $K > 2$                               |
|--------------------------|-----------------------------------------------------|---------------------------------------|
| <i>Entropy</i>           | $p \log \frac{1}{p} + (1 - p) \log \frac{1}{1 - p}$ | $\sum_{i=1}^k p_i \log \frac{1}{p_i}$ |
| <i>Gini Index</i>        | $2p(1 - p)$                                         | $\sum_{i,j} p_i p_j = 1 - \ p\ ^2$    |
| <i>“Simplest of all”</i> | $\min\{p, 1 - p\}$                                  | $1 - \max_i p_i = 1 - \ p\ _\infty$   |

# Decision Trees

## C4.5: Continuous attributes

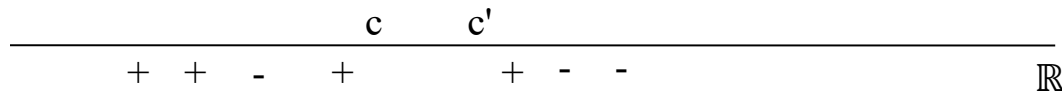
- Define a new discrete-valued attribute to test continuous attribute
  - partition the continuous value into a discrete set of intervals
  - $A_c = \text{true}$  iff  $A < c$
- How to select the best  $c$ ?

By using **information gain**!

# Decision Trees

## C4.5: Continuous attributes

- Testing all possible  $c$  values is **impossible** ...
- and **unnecessary**, since there would be no difference between two values  $c$  and  $c'$  when no training example takes an intermediate value for attribute  $A$ .



- Even more, a theorem reported in the text on CART written by Breiman et al. (1984) proves that:  
if  $k$  is the number of distinct values for  $A$ ,  
 $S_A = \langle x_1, x_2, \dots, x_k \rangle$  is the **ordered** list of distinct values of  $A$ ,  
**the best binary split** is one of  $k-1$  partitions  $\{x_1, \dots, x_h\}$  and  $\{x_{h+1}, \dots, x_k\}$  such that  $x_h$  and  $x_{h+1}$  have **different classes**.

# Decision Trees

## C4.5: Continuous attributes

- Therefore, the procedure is the following:
  - Sort the examples according to their values for A
  - For each ordered pair  $X_i, X_{i+1}$  in the sorted list, If the classes of  $X_i$  and  $X_{i+1}$  are different, then use the midpoint between their values as a candidate threshold.
  - Select the best candidate according to information gain.

### Example:

|            |    |      |     |      |     |     |    |
|------------|----|------|-----|------|-----|-----|----|
| Value:     | 10 | 15   | 21  | 28   | 32  | 40  | 50 |
| Class:     | No | Yes  | Yes | No   | Yes | Yes | No |
| Threshold: |    | 12.5 |     | 24.5 | 30  |     | 45 |

# Decision Trees

## Dealing with Missing Values

Problem: **How to deal with missing values of A?**

Some solutions:

- *Off-line*: face the problem during the data transformation step of the KDD process.
  - Give the example the most common value for the attribute among the training examples that reached the same node in the decision tree
  - Discard examples or attributes with null values.
- *On-line*: modify the decision tree induction algorithm in order to deal with missing values.

# Decision Trees

## Dealing with Missing Values

Problem: How to deal with missing values of A?

Issues related to the on-line approach:

- 1. Evaluating tests.** Selection of a test on which to partition the training set is made on the basis of heuristic criteria such as gain and gain ratio. If two tests use attributes with different numbers of unknown values, how should this be taken into account when weighting their relative desirability?

# Decision Trees

## Dealing with Missing Values

Problem: How to deal with missing values of A?

Issues related to the on-line approach:

- 2. Partitioning the training set.** Once a test has been selected, training cases with unknown values of the relevant attribute cannot be associated with a particular outcome of the test, and so cannot be assigned to a particular subset  $T_i$ . How should these cases be treated in the partitioning?

# Decision Trees

## Dealing with Missing Values

Problem: How to deal with missing values of A?

Issues related to the on-line approach:

- 3. Classifying an unseen case.** When the decision tree is used to classify an unseen case, how should the system proceed if the case has an unknown value for the attribute tested in the current decision node?

# Decision Trees

## Issue 1: Evaluating tests

- Obviously, a test provides no information about the class membership of cases when the value of the test attribute is unknown.
- Therefore, if the value of  $A$  is known in fraction  $F$  of the cases in the training set then

$$G(S, t) = F \cdot \left( E(S) - \sum_i \frac{|S_i|}{|S|} E(S_i) \right) + (1 - F) \cdot 0$$

| Day | Outlook  | Temperature | Humidity | Wind   | PlayTennis |
|-----|----------|-------------|----------|--------|------------|
| D1  | Sunny    | Hot         | High     | Weak   | No         |
| D2  | Sunny    | Hot         | High     | Strong | No         |
| D3  | Overcast | Hot         | High     | Weak   | Yes        |
| D4  | Rain     | Mild        | High     | Weak   | Yes        |
| D5  | Rain     | Cool        | Normal   | Weak   | Yes        |
| D6  | Rain     | Cool        | Normal   | Strong | No         |
| D7  | Overcast | Cool        | Normal   | Strong | Yes        |
| D8  | Sunny    | Mild        | High     | Weak   | No         |
| D9  | Sunny    | Cool        | Normal   | Weak   | Yes        |
| D10 | Rain     | Mild        | Normal   | Weak   | Yes        |
| D11 | Sunny    | Mild        | Normal   | Strong | Yes        |
| D12 | ?        | Mild        | High     | Strong | Yes        |
| D13 | Overcast | Hot         | Normal   | Weak   | Yes        |
| D14 | Rain     | Mild        | High     | Strong | No         |

# Decision Trees

## Issue 1: Evaluating tests

- Restrict our attention to the case whose value for **outlook** is known

| Overcast | Play | Don't play | Total |
|----------|------|------------|-------|
| sunny    | 2    | 3          | 5     |
| overcast | 3    | 0          | 3     |
| rain     | 3    | 2          | 5     |
| Total    | 8    | 5          | 13    |

# Decision Trees

## Issue 1: Evaluating tests

13 training examples: 8 are Yes, 5 are No

$$E(S) = -8/13 \log_2(8/13) - 5/13 \log_2(5/13) = .961$$

The entropy of Outlook is:

$$E(\text{Outlook}) = 5/13(-2/5 \log_2(2/5) - 3/5 \log_2(3/5)) + \dots = 0.747$$

The information gain we obtained by performing a test on *Outlook* is ( $F=13/14$ ):

$$G(\text{Outlook}) = 13/14(0.961 - 0.747) = 0.199$$

*The gain is slightly lower than the previous one (0.247)*

# Decision Trees

## Issue 2: Partitioning the training set

### A probabilistic approach:

Suppose the test  $t$  has  $z$  outcomes  $v_1, v_2, \dots, v_z$ , which partition  $S$  into  $z$  subsets  $S_1, S_2, \dots, S_z$ .



Idea: Associate each case in each subset  $S_i$  with a weight representing the probability that the case belongs to the subset:

- If the case has a known outcome this weight is 1
- otherwise the weight is just the probability of outcome  $v_i$  at this point

$|S_i|$  = Sum of the fractional weights of the cases

Therefore, a case from  $S$  with weight  $w$  whose outcome is not known is assigned to each subset  $S_i$  with weight

$$w * \text{probability of outcome } v_i$$

# Decision Trees

## Issue 2: Partitioning the training set

where *probability of outcome*  $v_i =$

*= sum of the weights of cases in  $S$  known to have outcome  $v_i$  /  
sum of the weights of the cases in  $S$  with known outcomes  
on this test.*

Example:

The case

| Outlook | Temp. | Humidity | Wind   | PlayTennis |
|---------|-------|----------|--------|------------|
| ?       | Mild  | High     | Strong | Yes        |

is assigned to all blocks of the partition with weights

sunny: 5/13

overcast: 3/13

rain: 5/13

# Decision Trees

## Issue 2: Partitioning the training set

- Focus on subset *outlook=sunny*

| Day | Outlook | Temp. | Humidity | Wind   | Play Tennis | Weight |
|-----|---------|-------|----------|--------|-------------|--------|
| D1  | Sunny   | Hot   | High     | Weak   | No          | 1      |
| D2  | Sunny   | Hot   | High     | Strong | No          | 1      |
| D8  | Sunny   | Mild  | High     | Weak   | No          | 1      |
| D9  | Sunny   | Cold  | Normal   | Weak   | Yes         | 1      |
| D11 | Sunny   | Mild  | Normal   | Strong | Yes         | 1      |
| D12 | ?       | Mild  | High     | Strong | Yes         | 5/13   |

# Decision Trees

## Issue 2: Partitioning the training set

- If this subset is partitioned further by the test on *Humidity*, the class distributions are
  - Humidity = High    5/13 class **Play**, 3 class **Don't Play**
  - Humidity = Normal    2 class **Play**, 0 class **Don't play**

For the first subset there is no test that improves on the situation.

# Decision Trees

## Issue 2: Partitioning the training set

- Similarly, the subset corresponding to *outlook=rain* cannot be partitioned into single-class subset. Therefore, the decision tree has the following structure:

Outlook=sunny:

| Humidity=normal: Play (2.0)

| Humidity=high: Don't Play (3.4/0.4)

Outlook=overcast: Play (3.2)

Outlook=rain:

| Wind=strong: Don't Play (2.4/0.4)

| Wind=weak: Play (2.0)

# Decision Trees

## Issue 2: Partitioning the training set

- The numbers at leaves, of the form (N) or (N/E) should be interpreted as follows:
  - N=sum of the fractional cases that reach the leaf
  - E= sum of cases that belong to other class than the nominated class

Example:

**Don't Play(3.4/0.4)**

Mean that **3.4** training cases reached this leaf, of which **0.4** did not belong to class **Don't Play**.

# Decision Trees

## Issue 3: Classifying unseen cases

- If a decision node is encountered at which the relevant attribute value is unknown the system explores all possible outcomes and combines the resulting classifications arithmetically.
- A *classification* is a class distribution rather than a single class
- The class with the highest probability is assigned as *the* predicted class.

# Decision Trees

## Dealing with Unknown Attributes: Classifying

Example: consider the unseen case

| Outlook | Temp. | Humidity | Wind |
|---------|-------|----------|------|
| Sunny   | Hot   | ?        | Weak |

- Outlook=sunny => first subtree
- Humidity=?
  - Humidity=Normal => **Play** with probability 2/2 (100%)
  - Humidity=High => **Don't Play** with probability 3/3.4(88%), **Play** with probability 0.4/3.4(12%)

We have  $2.0 + 3.4 = 5.4$  cases

$\Rightarrow$  **Play**:  $2.0/5.4 * 100\% + 3.4/5.4 * 12\% = 44\%$

$\Rightarrow$  **Don't Play**:  $3.4/5.4 * 88\% = 56\%$

# Data Mining Methods

## Decision Trees

The decision tree induction technique is very **efficient** in terms of processing time and provides a very **intuitive** method of analyzing the results. This intuitiveness, especially when contrasted with the less accessible output from neural networks, explains why decision trees are currently attracting much attention.

Decision trees have several **drawbacks**, however.

# Decision Trees

## Drawbacks: Optimality

- Current decision tree induction methods are not optimal. During the formation of a decision tree, once the algorithm makes a decision about the basis on which to split the node, that decision is never revised.

Example: Once the algorithm has decided that “**Tenure > 2.5 years**” is the most influential factor, it did not look for any new evidence on which to revise the decision. This loss of revision is due to the absence of backtracking, something for which most new most neural networks make provision.

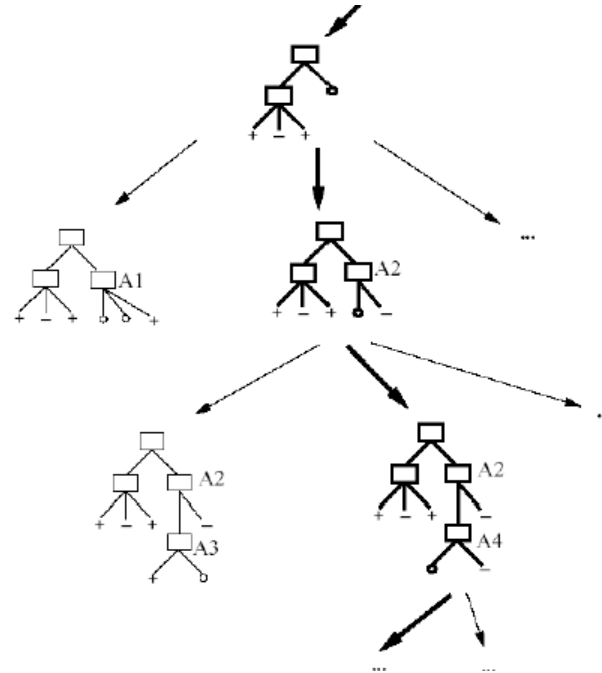
Methods for the construction of an optimal decision tree have exponential complexity.

# Decision Trees

## Hypothesis Space searched by C4.5

C4.5

- representation  
: trees
- scoring  
: entropy
- search  
: greedy



# Decision Trees

## Drawbacks: Error propagation

- Decision trees suffer from a problem of errors propagating throughout a tree
  - A very serious problem as the number of classes increases
- Since decision trees work by a series of local decisions, what happens when one of these local decisions is wrong?
  - Every decision from that point on may be wrong
  - We may never return to the correct path of the tree

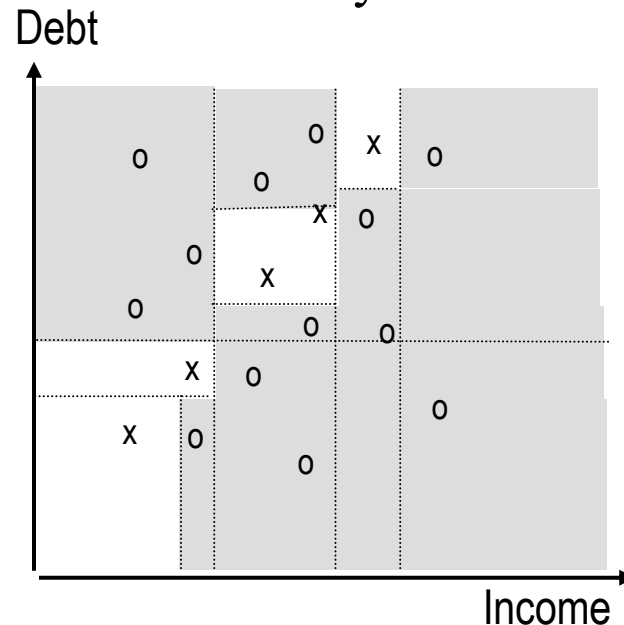
# Decision Trees

## Drawbacks: Fragmentation

- Decision trees are limited to problems that can be solved by dividing the solution space into successively smaller rectangles.

### Example

A decision tree representation, using univariate (single-field) node splits, partitions the input space into hyperplanes which are parallel to the attribute axes. Such a decision tree method cannot discover from data the formula  $x=y$  no matter how much training data it is given.



# Decision Trees

## Drawbacks: Fragmentation

In general, decision trees suffer from **fragmentation**.

When the tree has many layers of nodes, the amount of data that passes through the lower leaves is so small that it is difficult to make accurate decision on the class associated to each leaf.

To minimize fragmentation, the analyst can **prune** or trim back some of the lower leaves and nodes to effectively collapse some of the tree.

The result is an improved model where the real patterns rather than the noise (**overfitting** avoidance) are revealed and it is **simpler** to understand.

# Occam's Razor

*Entia non sunt multiplicanda praeter necessitatem*

**“Entities should not be multiplied beyond necessity”**



*William of Ockham (c.1285–1349)*

Occam's Razor has become a basic perspective for those who follow the scientific method. It is important to note that it is a heuristic argument that does not necessarily give correct answers; it is a loose guide to choosing the scientific hypothesis which (currently) contains the least number of unproven assumptions. Often, several hypotheses are equally "simple" and Occam's Razor does not express any preference in such cases.

# Occam's Razor

**Prefer shorter hypotheses!!**

Why prefer short hypotheses?

Argument in favor:

- Fewer short hypotheses than long hypotheses
- A short hypothesis that fits the data is unlikely to be a coincidence
- A long hypothesis that fits the data might be a coincidence

Argument opposed:

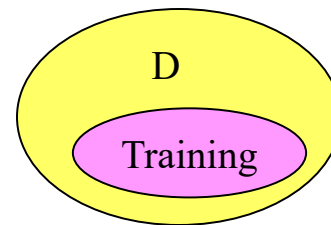
- There are many ways to define short hypotheses
  - E.g. Number of leaves, number of nodes, average tree depth, etc.

# Overfitting

$H$  is the space of hypotheses (e.g., set of decision trees built on a given set of attributes).

Consider error of hypothesis  $h \in H$  over

- Training data:  $\text{error}_{\text{training}}(h)$
- Entire distribution  $D$  of data:  $\text{error}_D(h)$



Hypothesis  $h \in H$  *overfits* training data if there is an alternative hypothesis  $h' \in H$  such that

$$\text{error}_{\text{training}}(h) < \text{error}_{\text{training}}(h') \quad \text{and}$$

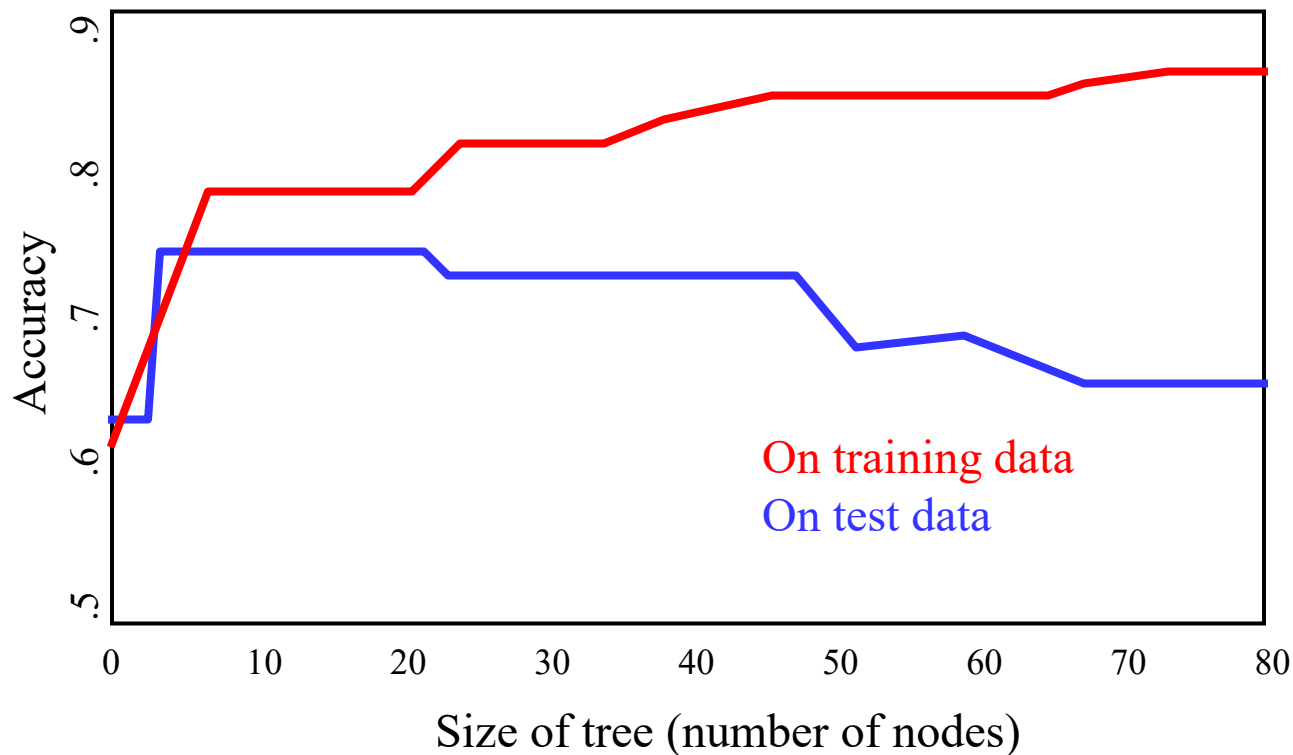
$$\text{error}_D(h) > \text{error}_D(h')$$

A hypothesis,  $h$ , is said to overfit the training set when there exists a hypothesis,  $h'$ , that outperforms  $h$  on the total distribution of instances.

# How can such happen?

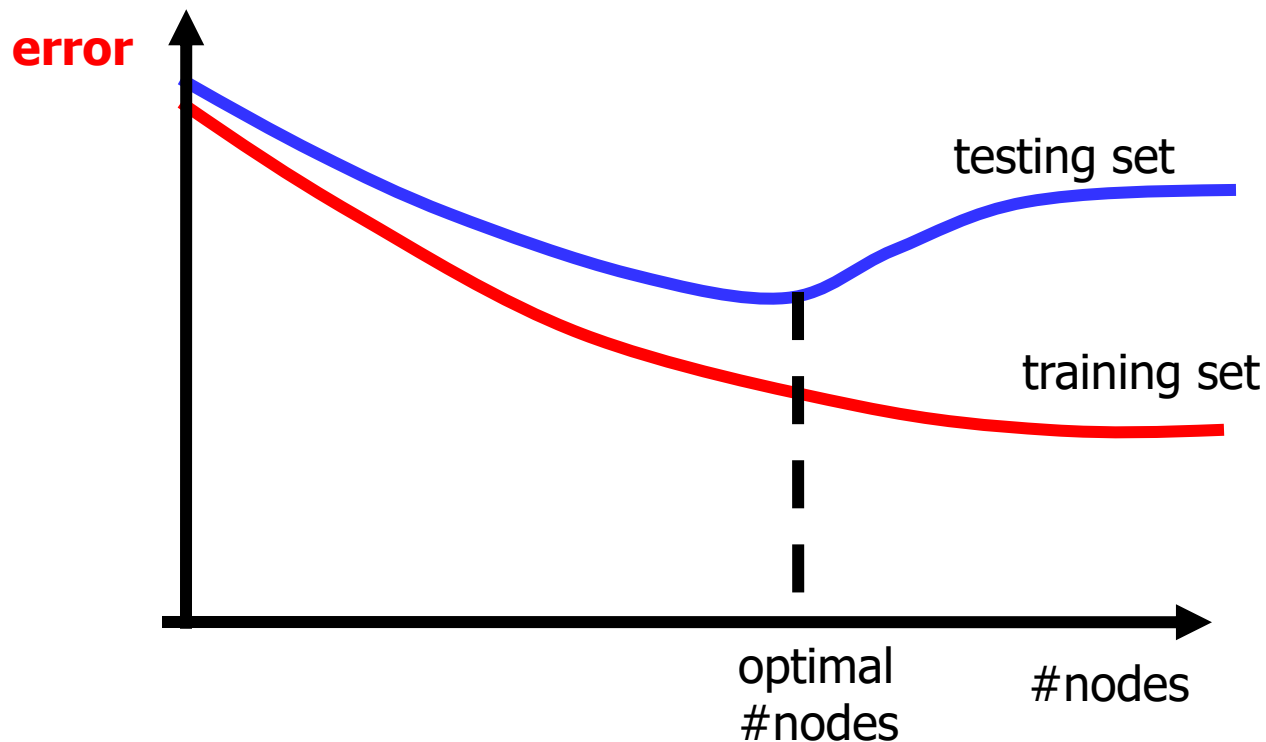
- One reason: noise
  - noisy data creates a large tree  $h$
  - $h'$  not fitting it is likely to work better
- Small samples
  - coincidences possible
  - attributes unrelated to target attribute (the class) *may* partition training data well

# Overfitting in Decision Tree Learning



**accuracy = 1 - error**

# Overfitting in Decision Tree Learning



# Decision Trees

## Overfitting avoidance

- How can we avoid overfitting?
- **Stop growing** the tree when
  - data split is not statistically significant, that is, there is not enough data to make reliable decisions, or
  - when the examples are acceptably homogenous
- Problem: it's hard to know when to stop!
- **Pruning**
  - Grow the full tree, then remove nodes for which there is not sufficient evidence

# Decision Trees

## Pruning Methods

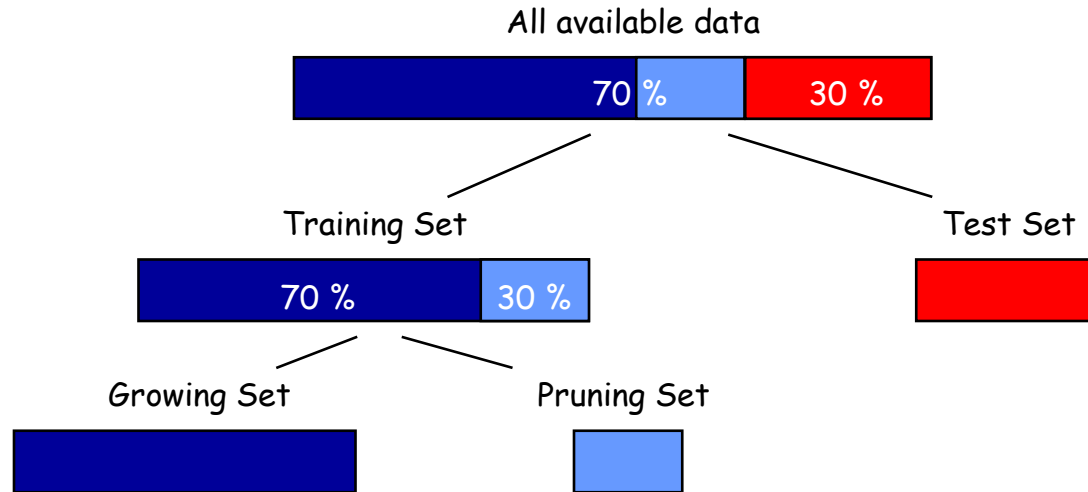


- What criterion to use?
  - **separate set** to evaluate the use of pruning
  - **use all data**, apply **statistical test** to estimate if expanding/pruning is likely to produce improvement
  - use an explicit **complexity measure** (coding length of data & tree), stop growth when minimized

# Data Mining Methods

## Training set vs. validation/testing set

Typical proportions



Problem with using “Pruning Set”: less data for “Growing Set”

# Decision Trees

## Overview of Selected Pruning Methods

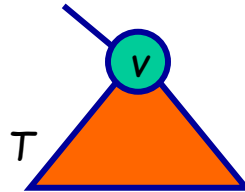
- **Reduced error pruning (REP)**  
Quinlan 87, Mingers 87, Esposito *et al.* 97, Elomaa & Kaariainen 2001
- **Minimal error pruning (MEP)**  
Niblett & Bratko 86, Cestnik & Bratko 91
- **Pessimistic error pruning (PEP)**  
Quinlan 87
- **Error-Based Pruning (EBP)**  
Quinlan 87
- **Cost-Complexity Pruning (CCP)**  
Brieman *et al.* 84

# Decision Trees

## Reduced Error Pruning

Use a **pruning set** to estimate accuracy of sub-trees and accuracy at individual nodes

Let  $T$  be a sub-tree rooted at node  $v$



Define

Gain from pruning at  $v = \# \text{misclassifications in } T - \# \text{misclassifications at } v$   
*#misclassifications is evaluated on the pruning set*

- prune  $T$  at node  $v$  if the gain is positive.
- “**Bottom-up restriction**”:  $T$  can only be pruned if it does not contain a sub-tree with lower error than  $T$

# Decision Trees

## Reduced Error Pruning

### Algorithm:

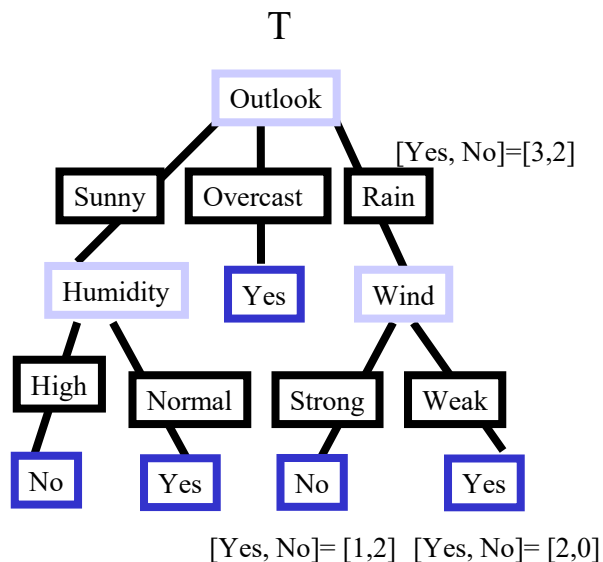
1. Build full decision tree from growing set
2. Visit the tree in post-order
  1. For every non-leaf node N
    - Prune sub-tree rooted by N, replace with majority class w.r.t. the growing set
    - Test accuracy of pruned tree on pruning set
  2. Remove the sub-tree that results in greatest improvement in accuracy on pruning set

# Decision Trees

## Reduced Error Pruning

**Example:** Let us consider

the tree T built from the growing set S

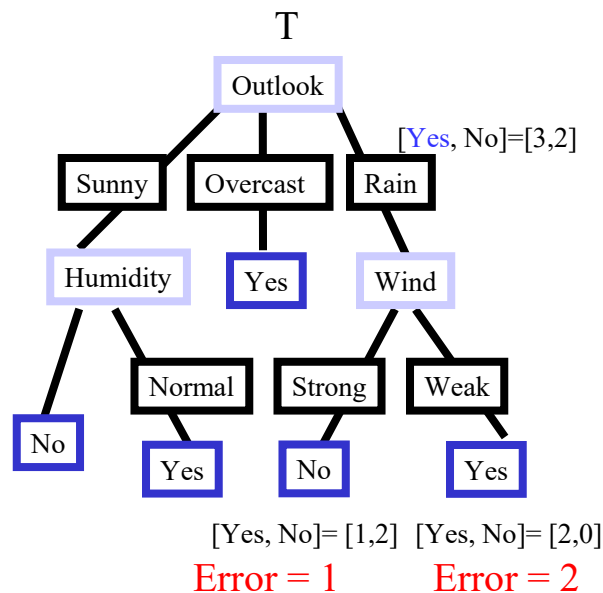


| Day | Outlook  | Temp | Humidity | Wind   | Play |
|-----|----------|------|----------|--------|------|
| D1  | Sunny    | Hot  | High     | Weak   | No   |
| D2  | Sunny    | Hot  | High     | Strong | No   |
| D3  | Overcast | Hot  | High     | Weak   | Yes  |
| D4  | Rain     | Mild | High     | Weak   | Yes  |
| D5  | Rain     | Cool | Normal   | Weak   | Yes  |
| D6  | Rain     | Cool | Normal   | Strong | No   |
| D7  | Overcast | Cool | Normal   | Weak   | Yes  |
| D8  | Sunny    | Mild | High     | Weak   | No   |
| D9  | Sunny    | Cold | Normal   | Weak   | Yes  |
| D10 | Rain     | Mild | Normal   | Strong | Yes  |
| D11 | Sunny    | Mild | Normal   | Strong | Yes  |
| D12 | Overcast | Mild | High     | Strong | Yes  |
| D13 | Overcast | Hot  | Normal   | Weak   | Yes  |
| D14 | Rain     | Mild | High     | Strong | No   |

# Decision Trees

## Reduced Error Pruning

We try to to prune the sub tree rooted in *Wind* and evaluate to effect of pruning according to the pruning set P



$$\text{Misclassification Error}(T_{\text{WIND}}, P) = 1 + 2 = 3$$

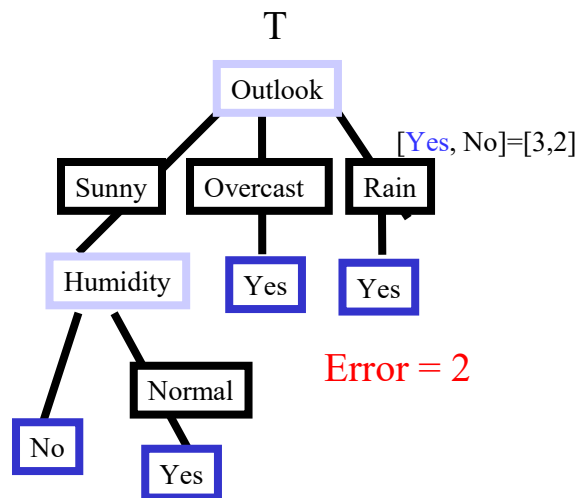
P

| Day        | Outlook     | Temp        | Humidity      | Wind          | Play       |
|------------|-------------|-------------|---------------|---------------|------------|
| D15        | Sunny       | Hot         | High          | Weak          | No         |
| D16        | Sunny       | Hot         | High          | Strong        | No         |
| <b>D17</b> | <b>Rain</b> | <b>Hot</b>  | <b>High</b>   | <b>Weak</b>   | <b>No</b>  |
| <b>D18</b> | <b>Rain</b> | <b>Mild</b> | <b>High</b>   | <b>Weak</b>   | <b>No</b>  |
| D19        | Rain        | Cool        | Normal        | Weak          | Yes        |
| <b>D20</b> | <b>Rain</b> | <b>Cool</b> | <b>Normal</b> | <b>Strong</b> | <b>Yes</b> |
| <b>D21</b> | <b>Rain</b> | <b>Cool</b> | <b>Normal</b> | <b>Weak</b>   | <b>Yes</b> |
| D22        | Overcast    | Cool        | Normal        | Strong        | No         |

# Decision Trees

## Reduced Error Pruning

We try to to prune the sub tree rooted in *Wind* and evaluate to effect of pruning according to the pruning set P



Misclassification Error( $T_{WIND \rightarrow leaf}, P$ )=2

Prune  $T_{WIND}$  since

$$\text{MisclassificationError}(T_{WIND \rightarrow leaf}, P) < \text{MisclassificationError}(T_{WIND}, P)$$

| Day        | Outlook     | Temp        | Humidity    | Wind        | Play      |
|------------|-------------|-------------|-------------|-------------|-----------|
| D15        | Sunny       | Hot         | High        | Weak        | No        |
| D16        | Sunny       | Hot         | High        | Strong      | No        |
| <b>D17</b> | <b>Rain</b> | <b>Hot</b>  | <b>High</b> | <b>Weak</b> | <b>No</b> |
| <b>D18</b> | <b>Rain</b> | <b>Mild</b> | <b>High</b> | <b>Weak</b> | <b>No</b> |
| D19        | Rain        | Cool        | Normal      | Weak        | Yes       |
| D20        | Rain        | Cool        | Normal      | Strong      | Yes       |
| D21        | Rain        | Cool        | Normal      | Weak        | Yes       |
| D22        | Overcast    | Cool        | Normal      | Strong      | No        |

# Instance Based Learning

**Idea:** We store all instances of the dataset  $\langle x, c(x) \rangle$

## **Classification:**

- Given a query instance  $x_q$
- We return: the concept (class) of the instance closest to  $x_q$  in the dataset

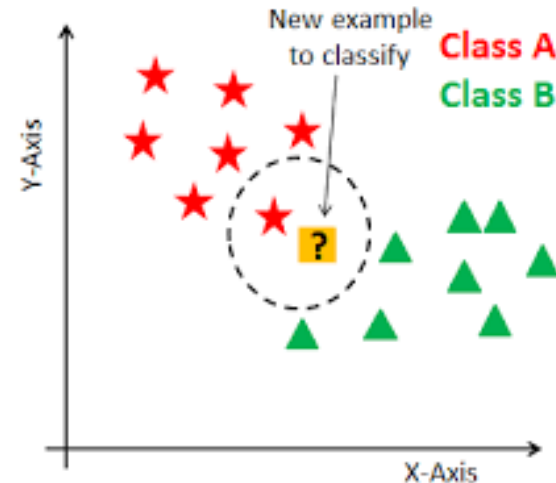
## **Justification:**

- The instance closest to  $x_q$  tends to have the same concept as  $f(x_q)$
- This does not hold when data is scarce

# Instance Based Learning: Nearest Neighbor

**Nearest Neighbor** is one of the simplest machine learning algorithms for classification.

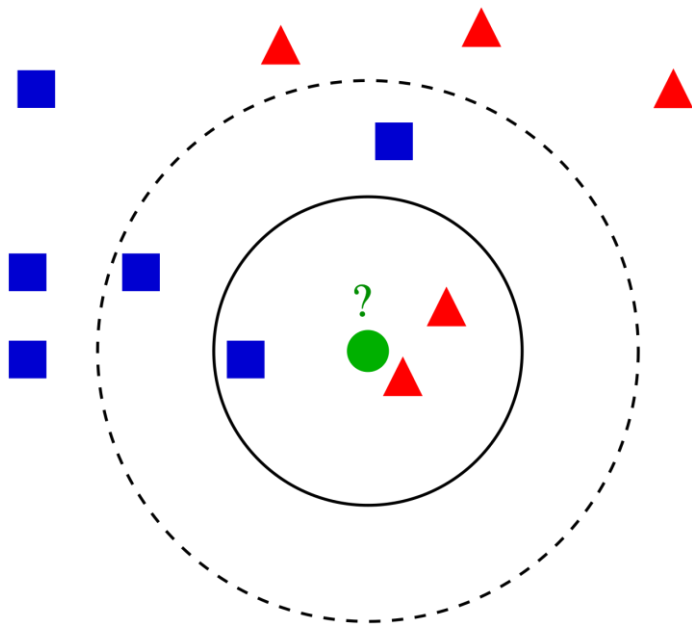
**Idea:** label the new point with the same label as the closest known point



# Instance Based Learning: K-Nearest Neighbor

- It classifies a new example by applying the **majority voting** technique, considering the k nearest neighbors
- **K** identifies the number of neighbors to consider
- Normally, **Euclidean distance** is used to identify the neighbors, but others can be used (e.g., Manhattan distance)

# Instance Based Learning: K-Nearest Neighbor



We want to label the green circle.

We use  $k=3$ :

- We select the 3 nearest neighbors 2 are red, 1 is blue
- Votes: 2 Red > 1 Blue

Therefore, the green point will be classified as a **red triangle**.

What if we used  $k=5$ ?

# The limits of a single model

Every supervised model faces the bias-variance tradeoff, which is inherently linked to its **capacity**:

High-Capacity Models (e.g., deep decision trees, large neural networks): have low bias as they fit the training data very closely. However, they suffer from high variance and instability; small fluctuations in the training set can produce drastically different models, leading to overfitting.

Low-Capacity Models (e.g., shallow trees, linear regression): are highly stable but often systematically wrong. They suffer from high bias because they are too simple to capture the underlying patterns of the data, leading to underfitting.

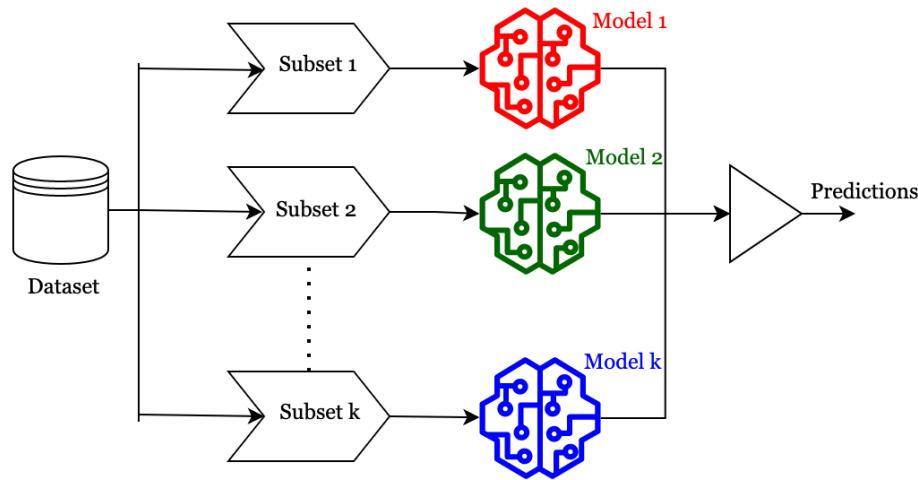
# The limits of a single model

## Ensemble Learning

Reducing bias tends to increase variance, and vice versa.

**No single model sits at the optimal point for both.**

Ensemble methods attack this problem directly: by combining multiple models, we can reduce variance (bagging), reduce bias (boosting), or both (stacking).



# Wisdom of the crowd

Three conditions must hold for an ensemble to outperform its members:

- **Diversity:** the models must make different errors. If all models fail on the same examples, combining them does not help.
- **Independence:** errors should not be correlated. This is why ensemble members are often trained on different subsets of data or features.
- **Aggregation:** the combination rule must be sensible and robust (majority vote, average, learned meta-model).

If the base learners are better than random and their errors are uncorrelated, the ensemble error decreases as we add more members.

# Voting: the simplest ensemble

A Voting Classifier trains **K independent base learners** on the same training set and combines their predictions at inference time. No model knows about the others.

**Hard voting:** each model casts a vote for a class. The class with the most votes wins.

**Soft voting:** each model outputs a probability vector. The vectors are averaged and the class with the highest average probability wins.

# Voting: the simplest ensemble

The key design choice is **diversity**: a Voting Classifier works best when the base learners are as different as possible.

Different algorithms (SVM, decision tree, KNN), different feature subsets, or different hyperparameters. If all models make the same errors, voting does not help.

# **Bagging: reducing variance through resampling**

Bagging (Bootstrap AGGregatING) trains  $K$  copies of the same base learner on  $K$  different bootstrap samples of the training set.

Each bootstrap sample is drawn with replacement and has the same size as the original dataset.

The final prediction is a simple average (regression) or majority vote (classification)

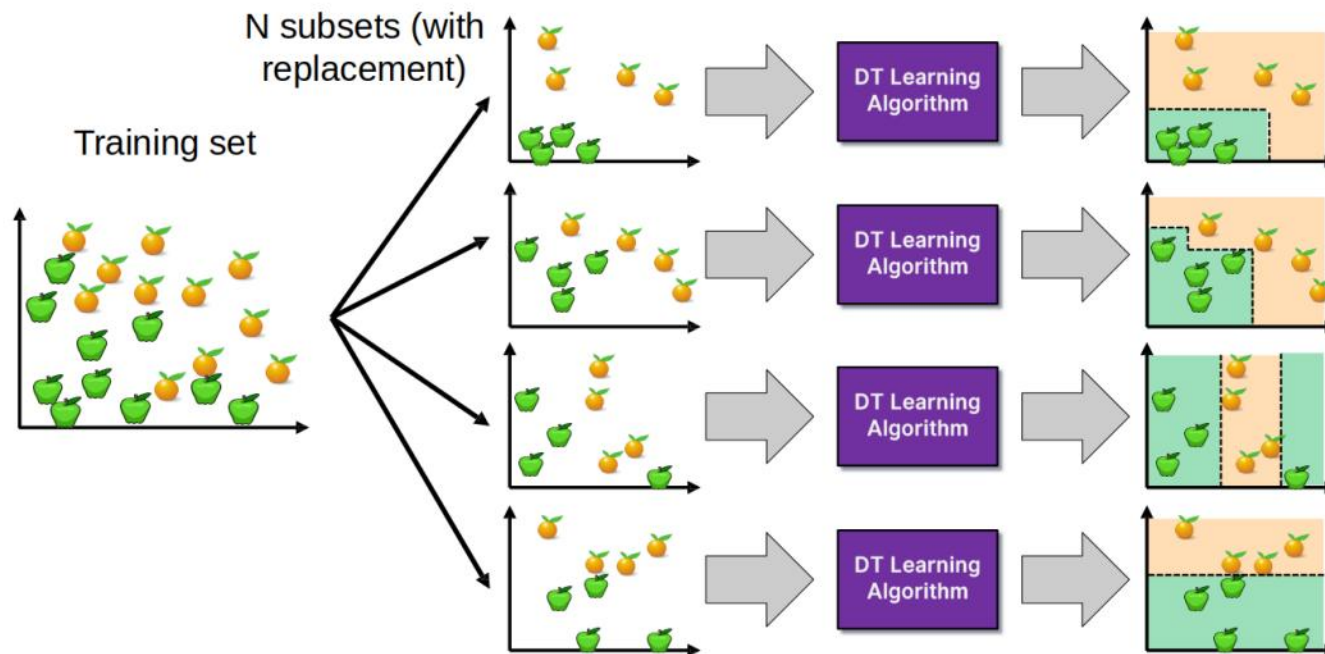
# Bagging: reducing variance through resampling

Because each model is trained on a different sample, the models make different errors.

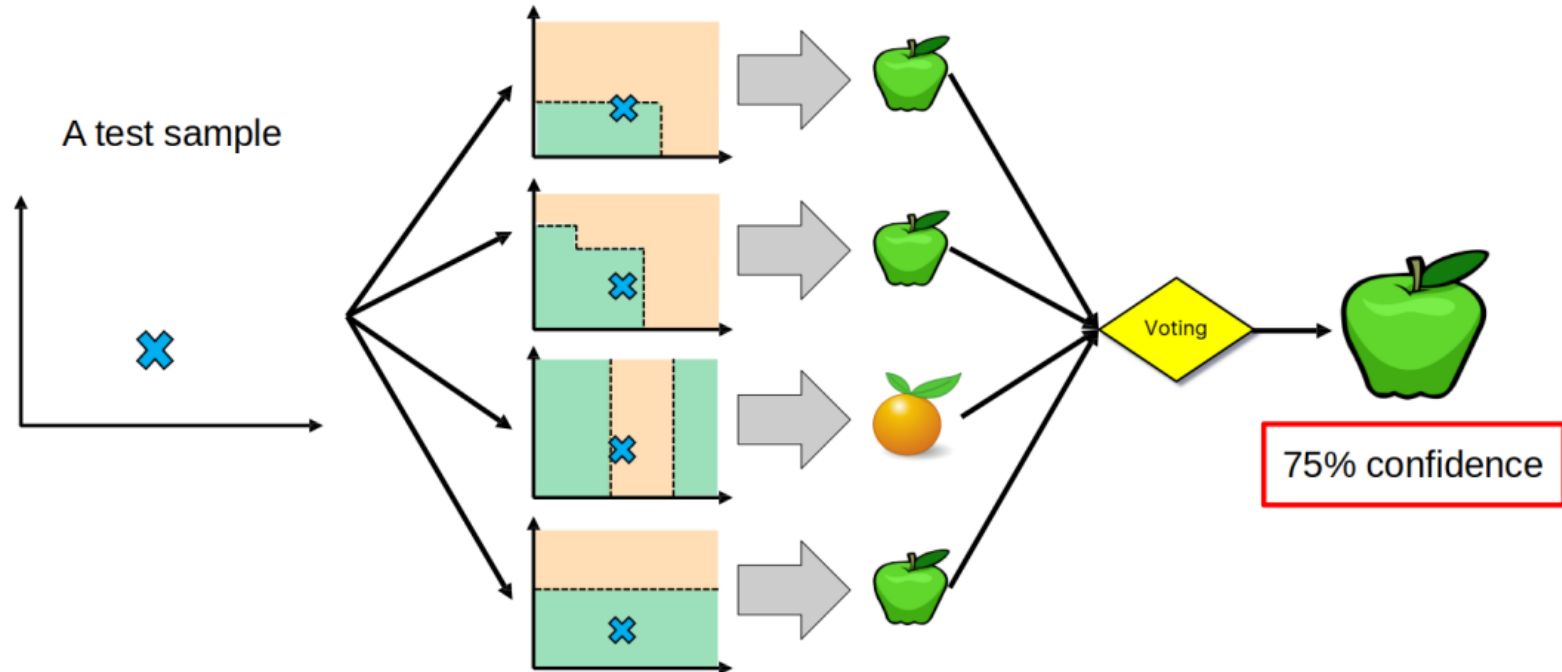
The idea is that averaging cancels out the individual errors and reduces variance so the ensemble is more stable than any single base learner.

**Random Forest** extends bagging with one additional trick: at each split, only a random subset of features is considered. This decorrelates the trees further and reduces variance even more.

# Random Forest



# Random Forest



# Bagging: limitations

Bagging reduces variance but **does not reduce bias**.

If the base learner is systematically wrong (e.g. a linear model on non-linear data), averaging many copies of it will not fix the problem.

For bias reduction, boosting is the solution.

# Boosting: learning from mistakes

Boosting trains base learners sequentially. Each new learner focuses on the examples that the previous learners got wrong.

- **AdaBoost** implements this idea by reweighting the training examples. After each round, the weight of misclassified examples is increased and the weight of correctly classified examples is decreased.

The next learner is trained on this reweighted distribution and is therefore forced to focus on the hard cases.

The final prediction is a weighted vote of all base learners, where each model's contribution is scaled by its accuracy-based weight

# Boosting: learning from mistakes

- **XGBoost** generalizes AdaBoost by framing boosting as gradient descent in function space. Instead of reweighting examples, it fits each new learner to the negative gradient of the loss function, effectively training models to predict the residual errors of the current ensemble.

To understand how XGBoost utilizes residuals, consider the task of predicting a person's actual age of 30 years ( $y = 30$ ):

1. Base Model ( $h_0$ ): Makes an initial crude estimate, predicting 20. The residual (error) is  $30 - 20 = 10$ .

# Boosting: learning from mistakes

2. Model 2 ( $h_1$ ): Trained specifically to predict that residual of 10. It predicts 8.

Updated Prediction: The ensemble now predicts  $20 + 8 = 28$ .

The new residual is  $30 - 28 = 2$ .

3. Model 3 ( $h_2$ ): Trained to predict the remaining residual of 2. It predicts 1.5.

Final Ensemble Prediction:  $28 + 1.5 = 29.5$ .

As more learners are added, the final output converges toward the target value through the summation of all individual predictions:

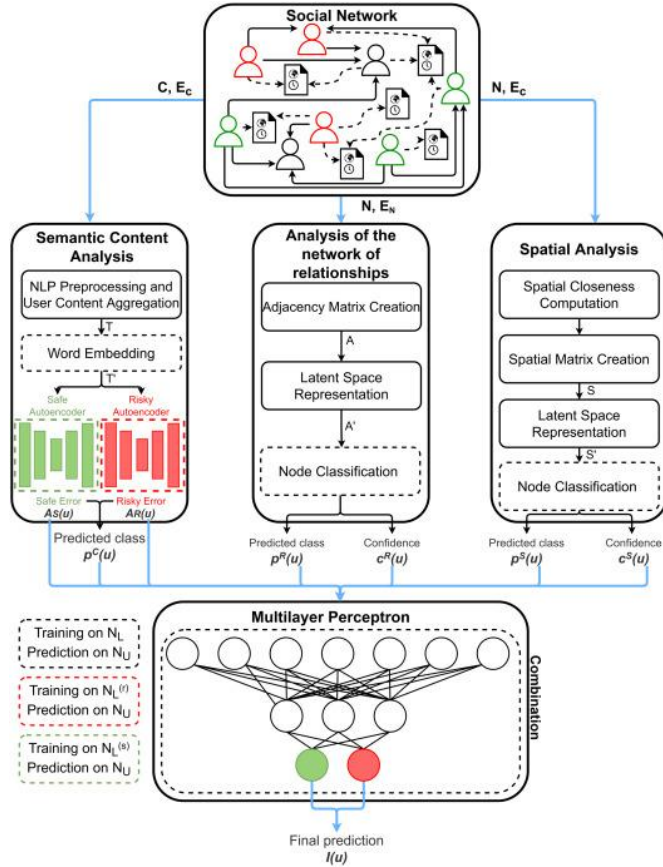
$$Y_{final} = \sum_{t=0}^T h_t(x)$$

# Stacking: Learning to Combine Models

**Stacking** addresses the bias-variance tradeoff by using a two-layer architecture:

- **Layer 0 (Base Models):** A set of **heterogeneous** models (e.g., a Random Forest, an SVM, and a KNN) are trained on the full dataset.
- **Layer 1 (Meta-Learner):** Instead of using a fixed rule like averaging, a meta-model (often a simple Linear Regression or Logistic Regression) is trained to combine the predictions of the base models.

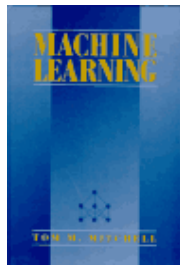
# Stacking: real example



Pellicani, A., Pio, G., Redavid, D., & Ceci, M. (2023). SAIRUS: Spatially-aware identification of risky users in social networks. *Information Fusion*, 92, 435-449.

<https://www.sciencedirect.com/science/article/pii/S1566253522002457>

# References

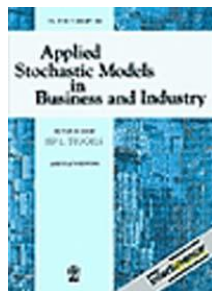


T. Mitchell  
*Machine Learning*  
Morgan Kaufmann, 1997  
Chapter: 3



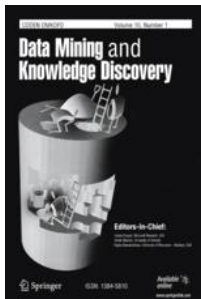
F. Esposito, D. Malerba, & G. Semeraro.  
A Comparative Analysis of Methods for Pruning Decision Trees.  
*IEEE Transactions on Pattern Analysis and Machine Intelligence*,  
PAMI-19, 5, pp. 476-491, 1997.

*Per approfondimenti:*



F. Esposito, D. Malerba, G. Semeraro & V. Tamma.  
The Effects of Pruning Methods on the Predictive Accuracy of  
Induced Decision Trees,  
*Applied Stochastic Models in Business and Industry* (formerly *Applied  
Stochastic Models in Data Analysis*), 15, 4, pp. 277-299, 1999.

# References



Sreerama K. Murthy.

Automatic Construction of Decision Trees from Data: A Multi-Disciplinary Survey.

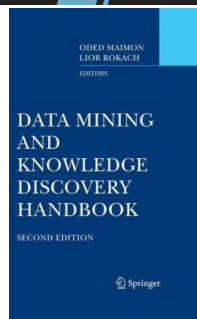
*Data Mining and Knowledge Discovery*, 2, pp. 345–389 (1998).



Leonard A. Breslow & David W. Aha

Simplifying Decision Trees: A Survey

*The Knowledge Engineering Review*, 12(1), pp. 1-40, 1997.



Lior Rokach & Oded Maimon

Decision Trees.

*Data Mining And Knowledge Discovery Handbook*, chapter 9, pp. 165-192, Springer, 2005.